Sequential Testing for A/B Experiments

A Practical Guide in Three Acts

Version 2

April 2026

Contents

Introduction **2**

1 Act 1 — The Peeking Problem **3**
1.1 The Motivation . . . 3
1.2 The Simulation . . . 3
1.3 Why Does This Happen? . . . 4
1.4 How Bad Is It? . . . 4

2 Act 2 — The Eppo Solution (Schmit & Miller, 2024) **5**
2.1 The Big Idea . . . 5
2.2 The Simulation . . . 6
2.3 The Pipeline, Step by Step . . . 7

3 Act 3 — What If You Don't Have Eppo? **10**
3.1 The Simulation . . . 10
3.2 Intuitive Explanation . . . 11
3.3 Method 1: Bonferroni Correction (Simplest) . . . 11
3.4 Method 2: Pocock Boundaries (1977) . . . 12
3.5 Method 3: O'Brien–Fleming Boundaries (1979) . . . 13
3.6 Head-to-Head Comparison . . . 15

4 Summary **16**

5 References **17**

A Python Code **18**
A.1 Simulation: The Peeking Problem . . . 18
A.2 Implementation: Bonferroni Correction . . . 19
A.3 Implementation: Pocock Boundaries . . . 20
A.4 Implementation: O'Brien–Fleming Boundaries . . . 22
A.5 Simulation: Comparing All Methods . . . 23

Introduction

The problem. Every company running A/B tests faces a temptation: peek at the results before the experiment is over. Traditional statistics breaks when you do this — the false positive rate inflates far beyond the promised 5%.

The solution. *Sequential testing* provides statistical methods that remain valid no matter when or how often you check. Modern platforms like Eppo build this into their pipeline automatically.

This guide. Three acts, each with a simulation, intuitive explanations, and key formulas with plain-language translations:

1. **The Peeking Problem** — Why checking early is dangerous.
2. **The Eppo Solution** — How a modern platform solves it.
3. **DIY Alternatives** — What to do if you don't have Eppo.

Target audience. People comfortable with algebra and basic probability. No calculus required. Every formula is accompanied by a plain-language translation.

What changed from v1: Removed subtitle from Act 3; added detailed implementation instructions with step-by-step equations for each DIY method; added Appendix A with Python code for simulation and implementation snippets.

1 Act 1 — The Peeking Problem

1.1 The Motivation

Intuitive Explanation

You work at a company running an A/B test. Half your users see the old website, half see a new design. Every day, you check the dashboard: "Is the new design better?" After a week, the dashboard shows $p = 0.03$ — that looks significant! You call it: the new design wins.

But here is the dirty secret of traditional statistics: **the more often you check, the more likely you are to see a false positive.** A test designed to have a 5% false positive rate can easily reach 20–30% if you peek at it repeatedly. One in four or five "winning" experiments might actually be noise.

This is not a theoretical concern. It is the single most common statistical mistake in online experimentation.

1.2 The Simulation

Simulation

What is being simulated: 1,000 A/B experiments where there is *no real effect* — any apparent difference is pure noise.

Two groups of observations (control and treatment) are generated from *identical* distributions. A standard t-test is computed after every new observation arrives.

A counter shows: "Number of times $p < 0.05$ so far." The user watches the p-value bounce around. Every time it dips below 0.05, it lights up red — a false positive.

Controls:

- How many total observations to simulate.
- How many independent experiments to run in parallel.
- Fixed seed toggle (deterministic replay).

A bar chart accumulates: "Fraction of experiments that showed $p < 0.05$ at *some* point during peeking." For 1,000 experiments with regular peeking, this fraction climbs to 20–30%.

A second mode checks only once, at the very end. That fraction stays near 5%.

Key takeaway: Peeking inflates false positives. The more you peek, the worse it gets.

1.3 Why Does This Happen?

Intuitive Explanation

Traditional statistical tests promise: "If there is no real effect, I will falsely cry wolf at most 5% of the time." But this promise comes with fine print: **you may only look at the result once, at a pre-determined time.**
Every time you peek, you get another chance for randomness to fool you. It is like rolling a die repeatedly — the more rolls, the more likely you are to get a six eventually, even though any single roll has only a 1/6 chance.

1.4 How Bad Is It?

Checking schedule	Actual false positive rate
Once at the end (as designed)	$\sim$5%
Daily for 1 week	$\sim$13%
Daily for 2 weeks	$\sim$19%
Daily for 4 weeks	$\sim$25%
Continuously (every observation)	Can exceed 30%

If you check a 4-week experiment every day, roughly 1 in 4 "significant" results could be false. This means you might ship features that have no real effect — or even hurt your metrics.

Key Takeaway

The peeking problem: Checking a traditional test repeatedly inflates the false positive rate far beyond its nominal level. We need a fundamentally different kind of test — one that remains valid no matter when or how often we look.
That kind of test exists. It is called a **sequential test**, and it is what Eppo (and similar platforms) use under the hood.

2 Act 2 — The Eppo Solution (Schmit & Miller, 2024)

2.1 The Big Idea

Intuitive Explanation

Eppo replaces the standard confidence interval with a **sequential confidence interval** — a band around the estimated treatment effect that is valid at *every moment* since the experiment started, not just at one pre-planned analysis time.
The key insight: a sequential CI is wider than a standard CI (you pay a small price for the freedom to peek), but the error guarantee holds no matter when you look. And Eppo reduces the noise in your data using pre-experiment information, making the CI narrower and experiments faster.

2.2 The Simulation

Simulation

What is being simulated: A single A/B experiment flowing through Eppo's pipeline, stage by stage.
Animated pipeline diagram. Data flows left-to-right through labelled stages. Each stage lights up as it is explained.
Stages (rendered as connected boxes):

$$\boxed{\text{Randomise}} \rightarrow \boxed{\text{Collect}} \rightarrow \boxed{\text{Adjust}} \rightarrow \boxed{\text{Estimate}} \rightarrow \boxed{\text{CI}} \rightarrow \boxed{\text{Decide}}$$

Step-by-step mode: A button labelled **"Step through pipeline"** pauses at each stage. At each stage, a distinct visual state is shown:

Stage	Visual state
Randomise	Users appear as dots, randomly coloured blue (control) or green (treatment).
Collect	Each dot acquires labels: a pre-experiment value X and an outcome Y. Tooltip on hover shows both values.
Adjust	Dots animate: $Y \rightarrow Y^*$. A regression line overlays each group. Dots shift vertically to their residual position. Spread visibly decreases.
Estimate	Group means appear as horizontal lines. The lift displays as a labelled gap between lines.
CI	A confidence interval band draws around the estimated lift. The band narrows as more data arrives.
Decide	The CI is colour-coded: **green** (entirely above 0), **red** (entirely below 0), or **grey** (crosses 0). A verdict label appears.

Side-by-side comparison mode: A button labelled **"Compare with / without adjustment"** splits the display into two panels using the *same data* — making the benefit of noise reduction immediately visible.
Controls:

- True effect size δ (0 to 10%).
- Sample size n (100 to 100,000).
- Pre-experiment correlation ρ (0 to 0.95): controls how much noise the adjustment removes.
- Fixed seed toggle (deterministic replay).
- "Run 1,000 experiments" mode (batch statistics).

Key takeaway: Each pipeline stage transforms the data in a specific, visible way. Regression adjustment visibly tightens the CI; removing it shows the cost.

2.3 The Pipeline, Step by Step

Step 1: Randomise

Each user arriving at the website is randomly assigned to **control** (existing experience) or **treatment** (new feature). Randomisation ensures the groups are comparable — any difference in outcomes is caused by the treatment, not by pre-existing differences.

Step 2: Collect data over time

As users interact with the product, two quantities are recorded for each user:

- Y_i: the **outcome** during the experiment (e.g. purchase amount, clicks, session length).
- X_i: a **pre-experiment covariate** — the same metric from before the experiment started (e.g. purchases in the prior 2 weeks).

Step 3: Regression adjustment (noise removal)

Users are wildly different — some buy €0, others €500. This noise drowns out the treatment effect. Eppo reduces this noise by using pre-experiment data.

For each group **separately**, fit a model predicting Y from X:

$$\hat{f}_{\text{control}}(X), \qquad \hat{f}_{\text{treatment}}(X)$$

Then compute the **adjusted outcome** for each user:

$$Y_i^* = Y_i - \hat{f}_g(X_i)$$

where g is the user's group (control or treatment).

> "Predict what each user would have done based on their history, then subtract that prediction. What's left is the part you *couldn't* have predicted — and that's where the treatment effect hides."
> This is a generalisation of a technique called CUPED (Deng et al., 2013). The stronger the correlation ρ between pre-experiment and experiment behaviour, the more noise is removed: $\text{Var}(Y^*) = \text{Var}(Y)(1-\rho^2)$. At $\rho = 0.7$, half the variance is eliminated — equivalent to doubling your traffic.

Safety rule: The covariate X must come from the *pre-experiment* period. Never use data that could be affected by the treatment.

Step 4: Estimate the treatment effect

Compute the adjusted group means and the **relative lift**:

$$\hat{\tau}(t) = \hat{\mu}_1(t) - \hat{\mu}_0(t), \qquad \text{Relative lift}(t) = \frac{\hat{\tau}(t)}{\hat{\mu}_0(t)}$$

Here $\hat{\mu}_0(t)$ and $\hat{\mu}_1(t)$ are the average adjusted outcomes in the control and treatment groups at time t. Their difference $\hat{\tau}(t)$ is the estimated treatment effect. Dividing by the control mean gives a percentage: "the treatment increased purchases by +3%." Relative lift is preferred because it is **scale-invariant** — a 3% lift means the same thing whether the baseline is €10 or €10,000.

Step 5: Estimate the variance

$$\hat{\sigma}^2_{\hat{\tau}}(t) = \frac{s_0^2(t)}{n_0(t)} + \frac{s_1^2(t)}{n_1(t)}$$

Here $s_g^2(t)$ is the sample variance (spread) of the adjusted outcomes in group g, and $n_g(t)$ is the number of users in that group so far. The variance is estimated from the data — not assumed known.

Step 6: Construct the sequential confidence interval

This is the key formula. Instead of a standard CI (which uses the constant $z_{\alpha/2} \approx 1.96$), Eppo uses a time-varying multiplier:

$$\boxed{\mathrm{CI}(t) = \hat{\tau}(t) \ \pm \ \hat{\sigma}_{\hat{\tau}}(t) \cdot \underbrace{\sqrt{\frac{n+\nu}{n} \cdot \log\frac{n+\nu}{\nu\alpha^2}}}_{\text{sequential multiplier}}}$$

Unpacking the formula:

- $\hat{\tau}(t)$: the estimated treatment effect at time t (from Step 4).
- $\hat{\sigma}_{\hat{\tau}}(t)$: the estimated standard error (from Step 5).
- $n = n_0(t) + n_1(t)$: total observations so far.
- α: significance level (e.g. 0.05 for a 95% CI).
- ν: a tuning parameter set from the expected total sample size M: $\nu = M \cdot \hat{\sigma}^2$. It controls *which* sample size the CI is tightest for.
- The "sequential multiplier" replaces the constant 1.96. It starts higher (e.g. $\sim$3.8 early on) and slowly decreases toward 1.96 — but never quite reaches it. This is the **price of peeking**.

This formula comes from the *confidence sequence* framework of Howard et al. (2021), specifically the Normal mixture boundary. The mathematical guarantee: the true treatment effect lies inside the CI at *all times simultaneously* with probability $\geq 1-\alpha$.

Step 7: Decide

What you see	Colour	Action
CI entirely above 0	Green	Positive effect. Ship the feature.
CI entirely below 0	Red	Negative effect. Revert.
CI crosses 0, experiment ongoing	Grey	Inconclusive. Keep collecting.
CI crosses 0, experiment ended	Grey	No significant effect. Decide pragmatically.

This decision is valid at whatever time it is made. No need to pre-specify an analysis time. This solves the peeking problem from Act 1.

Key Takeaway

The Eppo pipeline in one sentence: Randomise → collect outcomes Y and covariates X → regression-adjust (remove predictable noise) → estimate lift and variance → wrap in a sequential CI → decide when CI excludes zero.

What makes it work:

- **Framework:** Howard et al. (2021) confidence sequences.
- **Noise reduction:** Generalised CUPED via per-group regression.
- **Output:** Relative lift with a CI that is valid at any time.

3 Act 3 — What If You Don't Have Eppo?

Act 2 described Eppo's sophisticated pipeline. But many teams run experiments without a dedicated platform. This act presents three progressively better approaches to peek-safe testing that anyone can implement with a spreadsheet or a few lines of code — and compares them head-to-head with Eppo.

3.1 The Simulation

Simulation

What is being simulated: A single A/B experiment monitored at K pre-planned analysis times, showing four methods side-by-side.

Panel 1 — Four confidence bands: The same experiment, with four CI bands evolving over time:

- **Red (Bonferroni):** Widest. Simple but conservative.
- **Orange (Pocock):** Constant threshold, tighter than Bonferroni.
- **Blue (O'Brien–Fleming):** Starts very wide, narrows to near-classical at the final analysis.
- **Green (Eppo / Howard):** Continuous monitoring. No need to pre-specify K.

Vertical dashed lines mark the K planned analysis times. Between analyses, the group-sequential methods have no valid CI — only Eppo's band is defined everywhere.

Panel 2 — 10,000 experiments: Run 10,000 experiments under the null ($\delta = 0$). Counters show the false positive rate for each method:

- Classical CI with K peeks: $\gg 5\%$.
- Bonferroni: $\leq 5\%$ (often well below).
- Pocock: $\leq 5\%$ (closer to 5%).
- O'Brien–Fleming: $\leq 5\%$.
- Eppo: $\leq 5\%$.

Panel 3 — Efficiency comparison: Under a true effect ($\delta > 0$), show average stopping time for each method.

Controls:

- Number of planned peeks K (2 to 20).
- True effect size δ (0 to 10%).
- Significance level α.
- Fixed seed toggle (deterministic replay).
- "Run 1,000 experiments" mode.

Key takeaway: All four methods control the false positive rate. The price of simplicity is wider intervals (slower detection). Even the simplest method (Bonferroni) is far better than naive peeking.

3.2 Intuitive Explanation

Intuitive Explanation

You want to check your experiment every Monday. If you use a standard 95% CI each time, you'll get false positives far too often (Act 1).
The simplest fix: **be stricter at each check.** If you plan to peek 4 times, demand stronger evidence each time. The three methods below differ in *how* they distribute that strictness across the peeks.
Think of it as a budget. You have a total error budget of $\alpha = 5\%$.

- **Bonferroni** distributes the budget equally: α/K per peek.
- **Pocock** distributes it more cleverly (using the correlation between test statistics at different peeks), but still equally across peeks.
- **O'Brien–Fleming** front-loads the budget: almost no spending early, most of it saved for the final analysis.

Eppo's approach (Act 2) doesn't need a fixed budget at all — it works for *any* number of peeks, continuously.

3.3 Method 1: Bonferroni Correction (Simplest)

The idea

Before the experiment starts, commit to checking exactly K times (e.g. every Monday for 4 weeks: $K = 4$). At each check, use a significance level of α/K instead of α.

The recipe

1. Choose K (number of planned peeks) and α (e.g. 0.05).
2. At each peek $k = 1, \ldots, K$, compute the standard CI using the adjusted significance level:
$$\text{CI}_k = \hat{\tau}(t_k) \ \pm \ \hat{\sigma}_{\hat{\tau}}(t_k) \cdot z_{\alpha/(2K)}$$
where $\hat{\tau}(t_k)$ is the estimated treatment effect at peek k, $\hat{\sigma}_{\hat{\tau}}(t_k)$ is its standard error, and $z_{\alpha/(2K)}$ is the critical value from a Normal distribution at the adjusted significance level.
3. If the CI does not cross 0: stop and declare significance.
4. If $k = K$ and the CI crosses 0: no significant effect.

K (peeks)	α/K	$z_{\alpha/(2K)}$	CI multiplier vs. 1.96
1	0.050	1.96	1.00× (no correction)
2	0.025	2.24	1.14×
4	0.0125	2.50	1.28×
8	0.00625	2.73	1.39×
12	0.00417	2.87	1.46×
20	0.0025	3.02	1.54×
52	0.00096	3.29	1.68× (weekly for a year)

With 4 planned peeks, replace $z = 1.96$ with $z = 2.50$. Your CI is 28% wider at each peek, but you can safely check 4 times without inflating errors.

Implementation step by step

Given data arrays `control[]` and `treatment[]` at each scheduled peek:

1. **Compute group statistics:**

$$\bar{Y}_0 = \frac{1}{n_0}\sum_{i=1}^{n_0} Y_{0,i}, \qquad s_0^2 = \frac{1}{n_0 - 1}\sum_{i=1}^{n_0}(Y_{0,i} - \bar{Y}_0)^2$$

   and analogously for group 1.

2. **Compute the estimated treatment effect and its standard error:**

$$\hat{\tau} = \bar{Y}_1 - \bar{Y}_0, \qquad \mathrm{SE} = \sqrt{\frac{s_0^2}{n_0} + \frac{s_1^2}{n_1}}$$

3. **Look up the adjusted critical value:**

$$z^* = z_{\alpha/(2K)} = \Phi^{-1}\Big(1 - \tfrac{\alpha}{2K}\Big)$$

   where Φ^{-1} is the inverse of the standard Normal CDF (e.g. `scipy.stats.norm.ppf` in Python).

4. **Construct the CI and decide:**

$$\mathrm{CI} = \Big[\hat{\tau} - z^* \cdot \mathrm{SE},\ \hat{\tau} + z^* \cdot \mathrm{SE}\Big]$$

   If the CI does not contain 0, stop and declare significance. Otherwise, wait until the next scheduled peek.

Pros and cons

+ **Dead simple.** One line of code: change `1.96` to `z_alpha/(2K)`.
+ **Always valid.** Works for any test statistic, any distribution.
− **Conservative.** The CI is wider than necessary because Bonferroni ignores the correlation between test statistics at successive peeks.
− **Scales poorly.** As K grows, the correction becomes increasingly harsh.

3.4 Method 2: Pocock Boundaries (1977)

The idea

Like Bonferroni, pre-specify K equally-spaced analysis times. But instead of Bonferroni's crude α/K split, use a constant critical value c_P that accounts for the correlation between test statistics at different peeks.

Because the test statistics $Z_1, Z_2, \ldots, Z_K$ at successive analyses share overlapping data, they are positively correlated. Pocock's method exploits this to derive a tighter threshold.

The recipe

1. Choose K and α.
2. Look up (or compute) the Pocock critical value $c_P(K, \alpha)$.
3. At each peek k, compute the z-statistic: $Z_k = \hat{\tau}(t_k)/\hat{\sigma}_{\hat{\tau}}(t_k)$.
4. If $|Z_k| > c_P$: stop and declare significance.

K	c_P (Pocock)	$z_{\alpha/(2K)}$ (Bonf.)	Improvement
2	2.18	2.24	3% tighter
4	2.36	2.50	6% tighter
8	2.51	2.73	8% tighter
12	2.60	2.87	9% tighter
20	2.69	3.02	11% tighter

Pocock gives a constant threshold that is a few percent lower than Bonferroni's. The key advantage: the threshold is the same at every peek, so it is almost as simple to use as Bonferroni.

Implementation step by step

The Pocock critical value c_P is found by solving the following equation numerically. Let $\mathbf{Z} = (Z_1, \ldots, Z_K)$ be the vector of z-statistics at the K equally-spaced analyses. Under the null, these follow a multivariate Normal distribution with correlation matrix:

$$\mathrm{Cor}(Z_j, Z_k) = \sqrt{\frac{\min(j,k)}{\max(j,k)}} \qquad \text{for } 1 \le j, k \le K$$

The Pocock boundary c_P is the unique value satisfying:

$$\mathbb{P}\Big(\max_{1 \le k \le K} |Z_k| > c_P\Big) = \alpha$$

This is computed via multivariate Normal integration (see Appendix A for Python code using `scipy.stats.multivariate_normal`).

Once c_P is known, the decision rule at each peek is identical to Bonferroni, but with c_P replacing $z_{\alpha/(2K)}$:

1. At peek k, compute:
$$Z_k = \frac{\hat{\tau}(t_k)}{\mathrm{SE}(t_k)}$$
2. If $|Z_k| > c_P$: reject the null. Otherwise: continue.

3.5 Method 3: O'Brien–Fleming Boundaries (1979)

The idea

Instead of a constant threshold, use a threshold that *decreases* over time. At the first peek, demand very strong evidence; at the last peek, demand nearly the same evidence

as a classical test.

The recipe

At peek k (of K total), use the critical value:

$$c_k^{\text{OBF}} \approx z_{\alpha/2} \cdot \sqrt{K/k}$$

where $z_{\alpha/2} \approx 1.96$ is the standard Normal critical value.

Peek k (of K=4)	c_k^{OBF}	c_P (Pocock)	$z_{\alpha/(2K)}$ (Bonf.)	$z_{\alpha/2}$ (classical)
1 (25% of data)	4.05	2.36	2.50	1.96
2 (50% of data)	2.86	2.36	2.50	1.96
3 (75% of data)	2.34	2.36	2.50	1.96
4 (100% of data)	2.02	2.36	2.50	1.96

O'Brien–Fleming is extremely conservative early (threshold 4.05 at the first peek — you'd almost never stop that early) but barely penalises the final analysis (2.02 vs. 1.96 — only 3% wider).
This is the most efficient approach: if you run the experiment to completion, you pay almost nothing for the option to peek.

Implementation step by step

The O'Brien–Fleming boundary is the simplest to compute of the three "non-trivial" methods — it has a closed-form approximation.

1. **Before the experiment:** Fix K and α. At each peek k, the fraction of total data collected is $t_k = k/K$ (assuming equal spacing).
2. **At peek k, compute the critical value:**

   $$c_k^{\text{OBF}} = \frac{z_{\alpha/2}}{\sqrt{t_k}} = z_{\alpha/2} \cdot \sqrt{\frac{K}{k}}$$

   This is the Lan–DeMets spending function approximation to the original O'Brien–Fleming boundary.
3. **Compute the z-statistic** (same as Bonferroni and Pocock):

   $$Z_k = \frac{\hat{\tau}(t_k)}{\text{SE}(t_k)}$$

4. **Decide:** If $|Z_k| > c_k^{\text{OBF}}$: reject. Otherwise: continue to peek $k+1$.
5. **Convert to a confidence interval** (optional but recommended for interpretation):

   $$\text{CI}_k = \hat{\tau}(t_k) \pm c_k^{\text{OBF}} \cdot \text{SE}(t_k)$$

   Note: the CI width *changes* across peeks because c_k^{OBF} varies.

Practical tip: The exact O'Brien–Fleming boundaries (computed via multivariate Normal integration, like Pocock) differ slightly from the $z_{\alpha/2}/\sqrt{t_k}$ approximation. The approximation is accurate for $K \leq 10$ and is what most software implements. For larger K, use a library (e.g. `statsmodels.stats.multitest` or the R package `gsDesign`).

3.6 Head-to-Head Comparison

	Bonferroni	**Pocock**	**OBF**	**Eppo**
Pre-specify K?	Yes	Yes	Yes	No
Threshold type	Constant	Constant	Decreasing	Continuous
CI at peek 1 (K=4)	$2.50 \times \hat{\sigma}$	$2.36 \times \hat{\sigma}$	$4.05 \times \hat{\sigma}$	$\sim 3.8 \times \hat{\sigma}$
CI at final analysis (K=4)	$2.50 \times \hat{\sigma}$	$2.36 \times \hat{\sigma}$	$2.02 \times \hat{\sigma}$	$\sim 2.3 \times \hat{\sigma}$
Valid between peeks?	No	No	No	Yes
Variance reduction?	Manual	Manual	Manual	Built-in
Implementation	1 line	Lookup table	Formula	Platform

Key Takeaway

Which method should you use?

- **No platform, minimal effort:** Bonferroni. Commit to K peeks. Replace 1.96 with $z_{\alpha/(2K)}$. Done.
- **No platform, moderate effort:** O'Brien–Fleming. Nearly as efficient as Eppo at the final analysis, but requires pre-specifying K and cannot peek between scheduled analyses.
- **Need continuous monitoring or variance reduction:** Use a platform like Eppo. Continuous validity, regression adjustment, and no need to pre-commit to K.

The bottom line: even the crudest correction (Bonferroni) is *vastly* better than naive peeking. If you are checking results without any correction, you should start with Bonferroni *today*.

4 Summary

Act	Topic	Key message
1	The Peeking Problem	Checking a standard test repeatedly inflates false positives to 20–30%. You need a different kind of test.
2	The Eppo Solution	A sequential CI is valid at every moment. Regression adjustment removes noise, making experiments faster.
3	DIY Alternatives	Bonferroni (simplest), Pocock, O'Brien–Fleming — three ways to peek safely without a platform.

The core trade-off: if you want to peek, you must pay — your confidence interval must be wider. The question is how much wider, and the answer depends on how sophisticated a method you use:

Method	Price of peeking
No correction (naive)	Wrong answers (20–30% false positives)
Bonferroni	CI $\sim$28–68% wider (depending on K)
Pocock	CI $\sim$20–37% wider
O'Brien–Fleming	CI $\sim$3% wider at final analysis
Eppo (sequential CI)	CI $\sim$10–40% wider, valid *continuously*

5 References

1. **Deng, A., Xu, Y., Kohavi, R., & Walker, T.** (2013). Improving the sensitivity of online controlled experiments by utilizing pre-experiment data. *Proc. WSDM*, 123–132. `https://doi.org/10.1145/2433396.2433413` — The original CUPED paper. Variance reduction via pre-experiment covariates.

2. **Howard, S. R., Ramdas, A., McAuliffe, J., & Sekhon, J.** (2021). Time-uniform, nonparametric, nonasymptotic confidence sequences. *Annals of Statistics*, 49(2), 1055–1080. `https://doi.org/10.1214/20-AOS1991` — The confidence sequence framework that Eppo adopted.

3. **Pocock, S. J.** (1977). Group sequential methods in the design and analysis of clinical trials. *Biometrika*, 64(2), 191–199.

4. **O'Brien, P. C. & Fleming, T. R.** (1979). A multiple testing procedure for clinical trials. *Biometrics*, 35(3), 549–556.

5. **Schmit, M. & Miller, T.** (2024). Sequential confidence intervals for comparing two proportions with variance reduction. *Eppo Technical Report*. — Eppo's implementation combining Howard et al.'s confidence sequences with generalised CUPED.

A Python Code

This appendix provides self-contained Python code for:

1. A simulation demonstrating the peeking problem (Act 1).
2. Implementation snippets for each DIY method (Act 3).

All code requires `numpy` and `scipy`. Install with: `pip install numpy scipy`.

A.1 Simulation: The Peeking Problem

This simulation runs many A/B experiments under the null (no true effect) and measures how often naive peeking produces a false positive, versus checking only at the end.

```python
import numpy as np
from scipy import stats

def simulate_peeking(
    n_experiments: int = 10_000,
    n_obs_per_group: int = 500,
    peek_interval: int = 10,
    alpha: float = 0.05,
    seed: int = 42,
) -> dict:
    """Simulate the peeking problem under the null hypothesis.

    Returns a dict with false positive rates for:
      - 'naive_peeking': check every peek_interval observations
      - 'single_check':  check only at the end
    """
    rng = np.random.default_rng(seed)
    peek_times = list(range(peek_interval,
                            n_obs_per_group + 1,
                            peek_interval))

    false_pos_peek = 0
    false_pos_end = 0

    for _ in range(n_experiments):
        # Generate data from identical distributions (null)
        control = rng.normal(0, 1, n_obs_per_group)
        treatment = rng.normal(0, 1, n_obs_per_group)

        # --- Naive peeking ---
        ever_significant = False
        for t in peek_times:
            ctrl_t = control[:t]
            trt_t = treatment[:t]
            _, p_value = stats.ttest_ind(ctrl_t, trt_t)
            if p_value < alpha:
                ever_significant = True
                break  # stop at first significance
```

```python
        if ever_significant:
            false_pos_peek += 1

        # --- Single check at the end ---
        _, p_value_end = stats.ttest_ind(control, treatment)
        if p_value_end < alpha:
            false_pos_end += 1

    return {
        "naive_peeking": false_pos_peek / n_experiments,
        "single_check": false_pos_end / n_experiments,
        "n_experiments": n_experiments,
        "n_peeks": len(peek_times),
    }

# --- Run it ---
results = simulate_peeking()
print(f"False positive rate (single check): "
      f"{results['single_check']:.1%}")
print(f"False positive rate (peeking every "
      f"{10} obs, {results['n_peeks']} peeks): "
      f"{results['naive_peeking']:.1%}")
```

Expected output (approximately):

```
False positive rate (single check): 5.0%
False positive rate (peeking every 10 obs, 50 peeks): 27.3%
```

A.2 Implementation: Bonferroni Correction

```python
import numpy as np
from scipy import stats

def bonferroni_test(
    control: np.ndarray,
    treatment: np.ndarray,
    K: int,
    alpha: float = 0.05,
) -> dict:
    """Run a single-peek Bonferroni-corrected test.

    Call this function at each of the K scheduled peeks
    with the data collected so far.

    Returns:
      - 'reject': whether to reject the null
      - 'ci': the confidence interval for tau_hat
      - 'tau_hat': estimated treatment effect
      - 'z_star': adjusted critical value
    """
```

```python
    n0, n1 = len(control), len(treatment)
    tau_hat = treatment.mean() - control.mean()
    se = np.sqrt(control.var(ddof=1) / n0
                 + treatment.var(ddof=1) / n1)

    # Bonferroni-adjusted critical value
    z_star = stats.norm.ppf(1 - alpha / (2 * K))

    ci_lower = tau_hat - z_star * se
    ci_upper = tau_hat + z_star * se
    reject = ci_lower > 0 or ci_upper < 0

    return {
        "reject": reject,
        "ci": (ci_lower, ci_upper),
        "tau_hat": tau_hat,
        "z_star": z_star,
        "se": se,
    }

# --- Example usage ---
rng = np.random.default_rng(0)
K = 4  # plan to peek 4 times
N_total = 2000  # total obs per group
peek_every = N_total // K

control_all = rng.normal(10, 3, N_total)
treatment_all = rng.normal(10.3, 3, N_total)  # small effect

for k in range(1, K + 1):
    n = k * peek_every
    result = bonferroni_test(
        control_all[:n], treatment_all[:n], K=K
    )
    ci = result["ci"]
    status = "REJECT" if result["reject"] else "continue"
    print(f"Peek {k}/{K} (n={n}): "
          f"tau={result['tau_hat']:.3f}, "
          f"CI=[{ci[0]:.3f}, {ci[1]:.3f}], "
          f"z*={result['z_star']:.3f} -> {status}")
```

A.3 Implementation: Pocock Boundaries

```python
import numpy as np
from scipy import stats, optimize

def pocock_critical_value(
    K: int, alpha: float = 0.05, n_sim: int = 200_000,
    seed: int = 42,
) -> float:
```

```python
    """Find the Pocock boundary c_P by simulation.

    Simulates the multivariate Normal distribution of
    (Z_1, ..., Z_K) under the null and finds c_P such that
    P(max |Z_k| > c_P) = alpha.

    For production use, replace with exact multivariate
    Normal integration (e.g. mvnorm from R or PyMVPA).
    """
    rng = np.random.default_rng(seed)

    # Build the correlation matrix: Cor(Z_j, Z_k) = sqrt(min/max)
    fracs = np.arange(1, K + 1) / K
    corr = np.zeros((K, K))
    for j in range(K):
        for k in range(K):
            corr[j, k] = np.sqrt(
                min(fracs[j], fracs[k])
                / max(fracs[j], fracs[k])
            )

    # Simulate Z vectors
    samples = rng.multivariate_normal(
        np.zeros(K), corr, size=n_sim
    )
    max_abs_z = np.abs(samples).max(axis=1)

    # Find c_P: the (1 - alpha) quantile of max|Z|
    c_P = np.quantile(max_abs_z, 1 - alpha)
    return float(c_P)

def pocock_test(
    control: np.ndarray,
    treatment: np.ndarray,
    c_P: float,
) -> dict:
    """Test at a single peek using the Pocock boundary."""
    n0, n1 = len(control), len(treatment)
    tau_hat = treatment.mean() - control.mean()
    se = np.sqrt(control.var(ddof=1) / n0
                 + treatment.var(ddof=1) / n1)
    z_stat = tau_hat / se
    reject = abs(z_stat) > c_P

    ci_lower = tau_hat - c_P * se
    ci_upper = tau_hat + c_P * se

    return {
        "reject": reject,
        "z_stat": z_stat,
```

```
        "ci": (ci_lower, ci_upper),
        "tau_hat": tau_hat,
    }

# --- Example usage ---
K = 4
c_P = pocock_critical_value(K)
print(f"Pocock critical value for K={K}: {c_P:.3f}")

rng = np.random.default_rng(0)
N_total = 2000
peek_every = N_total // K

control_all = rng.normal(10, 3, N_total)
treatment_all = rng.normal(10.3, 3, N_total)

for k in range(1, K + 1):
    n = k * peek_every
    result = pocock_test(
        control_all[:n], treatment_all[:n], c_P=c_P
    )
    ci = result["ci"]
    status = "REJECT" if result["reject"] else "continue"
    print(f"Peek {k}/{K} (n={n}): "
          f"Z={result['z_stat']:.3f}, "
          f"CI=[{ci[0]:.3f}, {ci[1]:.3f}] -> {status}")
```

A.4 Implementation: O'Brien–Fleming Boundaries

```
import numpy as np
from scipy import stats

def obf_critical_value(k: int, K: int,
                       alpha: float = 0.05) -> float:
    """O'Brien-Fleming critical value at peek k of K.

    Uses the Lan-DeMets spending function approximation:
        c_k = z_{alpha/2} / sqrt(k / K)
    """
    z_alpha2 = stats.norm.ppf(1 - alpha / 2)
    t_k = k / K  # information fraction
    return z_alpha2 / np.sqrt(t_k)

def obf_test(
    control: np.ndarray,
    treatment: np.ndarray,
    k: int,
    K: int,
    alpha: float = 0.05,
) -> dict:
```

```python
    """Test at peek k of K using O'Brien-Fleming."""
    n0, n1 = len(control), len(treatment)
    tau_hat = treatment.mean() - control.mean()
    se = np.sqrt(control.var(ddof=1) / n0
                 + treatment.var(ddof=1) / n1)
    z_stat = tau_hat / se
    c_k = obf_critical_value(k, K, alpha)
    reject = abs(z_stat) > c_k

    ci_lower = tau_hat - c_k * se
    ci_upper = tau_hat + c_k * se

    return {
        "reject": reject,
        "z_stat": z_stat,
        "c_k": c_k,
        "ci": (ci_lower, ci_upper),
        "tau_hat": tau_hat,
    }

# --- Example usage ---
K = 4
rng = np.random.default_rng(0)
N_total = 2000
peek_every = N_total // K

control_all = rng.normal(10, 3, N_total)
treatment_all = rng.normal(10.3, 3, N_total)

for k in range(1, K + 1):
    n = k * peek_every
    result = obf_test(
        control_all[:n], treatment_all[:n], k=k, K=K
    )
    ci = result["ci"]
    status = "REJECT" if result["reject"] else "continue"
    print(f"Peek {k}/{K} (n={n}): "
          f"Z={result['z_stat']:.3f}, "
          f"c_k={result['c_k']:.3f}, "
          f"CI=[{ci[0]:.3f}, {ci[1]:.3f}] -> {status}")
```

A.5 Simulation: Comparing All Methods

This simulation runs all three methods (plus naive peeking) on the same experiments under the null, and reports their false positive rates.

```python
import numpy as np
from scipy import stats

def compare_methods(
```

```python
    n_experiments: int = 10_000,
    n_obs_per_group: int = 2000,
    K: int = 4,
    alpha: float = 0.05,
    seed: int = 42,
) -> dict:
    """Compare false positive rates of all methods."""
    rng = np.random.default_rng(seed)
    peek_every = n_obs_per_group // K

    # Pre-compute critical values
    z_bonf = stats.norm.ppf(1 - alpha / (2 * K))
    c_P = pocock_critical_value(K, alpha)

    counts = {
        "naive": 0,
        "bonferroni": 0,
        "pocock": 0,
        "obf": 0,
    }

    for _ in range(n_experiments):
        ctrl = rng.normal(0, 1, n_obs_per_group)
        trt = rng.normal(0, 1, n_obs_per_group)

        rej = {m: False for m in counts}

        for k in range(1, K + 1):
            n = k * peek_every
            c, t = ctrl[:n], trt[:n]
            tau = t.mean() - c.mean()
            se = np.sqrt(c.var(ddof=1)/n + t.var(ddof=1)/n)
            z = abs(tau / se)

            # Naive: standard z > 1.96
            if z > stats.norm.ppf(1 - alpha / 2):
                rej["naive"] = True

            # Bonferroni
            if z > z_bonf:
                rej["bonferroni"] = True

            # Pocock
            if z > c_P:
                rej["pocock"] = True

            # O'Brien-Fleming
            c_k = stats.norm.ppf(1 - alpha/2) / np.sqrt(k/K)
            if z > c_k:
                rej["obf"] = True

```

```python
        for m, rejected in rej.items():
            if rejected:
                counts[m] += 1

    return {m: c / n_experiments for m, c in counts.items()}

# --- Run it ---
rates = compare_methods(K=4)
print(f"{'Method':<15} {'FPR':>8}")
print("-" * 25)
for method, fpr in rates.items():
    label = method.upper().replace("_", " ")
    marker = " ***" if fpr > 0.06 else ""
    print(f"{label:<15} {fpr:>7.1%}{marker}")
```

Expected output (approximately):

```
Method              FPR
-------------------------
NAIVE            14.2% ***
BONFERRONI        3.1%
POCOCK            4.5%
OBF               4.8%
```

All corrected methods stay at or below 5%. Naive peeking exceeds it by a wide margin, even with only $K = 4$ peeks.